Less Tests More Power*

Sharing setup and selecting tests to make REST API test suites cheaper

Samuel van Nimwegen
University of Antwerp
s0225763

20 October 2026

Master's thesis in the LORE research group
Promotor: Prof. Serge Demeyer. Daily supervisor: Tolgahan Bardakci.

1 Thesis

1.1 Context

A REST API test sends HTTP requests to a running service and checks the answers with assertions. Most tests first send setup requests, such as a signup and a login, and then test one behaviour, such as placing an order. On a service that runs locally, a reset before every test puts the database back in its starting state, so tests stay independent. Black box coverage, which counts the parts of the API a suite reaches [1], can help indicate how strong a REST API test suite is. In the title, power means the strength of a REST API test suite, which black box coverage can help indicate but does not fully capture.

1.2 Problem Statement

REST API suites take time to run. One reason is that every test pays for its own setup, and on a local service for its own reset, even when many tests send the same setup requests. Another reason is that suites keep growing. A language model that amplifies a suite adds many tests [2, 3]. Suites run after every change, so this cost delays feedback and costs money in every build [4].

Sharing setup is one way to cut this cost, and it keeps every assertion. Testler [5] and TestBoost [6] do this for unit tests, and STICCER [7] merges database tests after removing some. None of them shares requests between black box REST API tests, where each test makes up its own values. Keeping fewer tests is another way, but a removed test takes its assertions with it. Reduced suites missed the failure in up to 52.2 percent of the failed builds of 32 Java projects [8], and Sartaj et al. note that it is still open how to minimise and select REST regression tests [4]. Boundary values, which are inputs at the edge of what the service accepts, could make the kept tests stronger [2]. Amplification

*Alternative titles for discussion:
(1) "Cut the Calls, Keep the Coverage: Cheaper REST API Test Suites"
(2) "Every Request Counts: Sharing and Selecting REST API Tests"
(3) "When Amplification Overshoots: Trimming Amplified REST API Test Suites"
(4) "Share the Setup, Select by Coverage: Cheaper REST API Test Suites"

adds new tests with such values [2, 3], but it does not change the values inside the tests a suite keeps.

Closing these gaps runs the same assertions sooner and cheaper in every build. It also shows a team how far a suite can be cut, and what edge values win back.

1.3 Research Questions

The main question is: how much cheaper can an existing REST API test suite become, and how much black box coverage does it lose? RQ1 covers sharing setup, RQ2 covers keeping fewer tests and boundary values, and RQ3 asks whether our tool pays for itself.

RQ1. How much does sharing setup cut requests, resets and run time, and does the suite keep its black box coverage and its assertions?

RQ2. When a suite keeps fewer tests, how much black box coverage does it lose, and how much do boundary values win back, compared with random valid values?

RQ3. How long does the tool take, and after how many runs does it pay back?

1.4 Challenge

Sharing setup and boundary values each bring their own challenges. For sharing, tests make up their own values, such as a fresh email, so the same signup looks different in every test. A REST service also keeps its state between requests. Two requests that look the same can differ, such as a login as another user, and a test that deletes data breaks a later test that reads it. A shared request can also let an assertion pass for the wrong reason, which a replay alone cannot show. For boundary values, a changed value must keep every assertion passing, so it can only be an edge that the service accepts. Such an edge often reaches the same operations and status codes as before.

2 Background

Over the past few years I have gained practical experience in REST API development and in software engineering in general, through projects at university and through my job. For the last two years I worked half time at TrackBox, where I did both software engineering and cloud engineering with microservices, Kubernetes and REST APIs. I really like working with REST APIs and with testing, so I was a big fan of this thesis topic. In the Software Testing course I studied boundary value testing, which this thesis builds on.

3 Method

We build a tool that sits between the tests and the service, like a proxy. It never changes the test code and never reads the service code. It works in five steps. RQ1 uses Record, Share and Replay. RQ2 uses Record, Select and Boundary values, without sharing.

- **Record:** The tool runs every test alone, after a reset where the service has one, stores each request and answer, and notes each passing assertion. Tests that fail alone stay in and run alone.

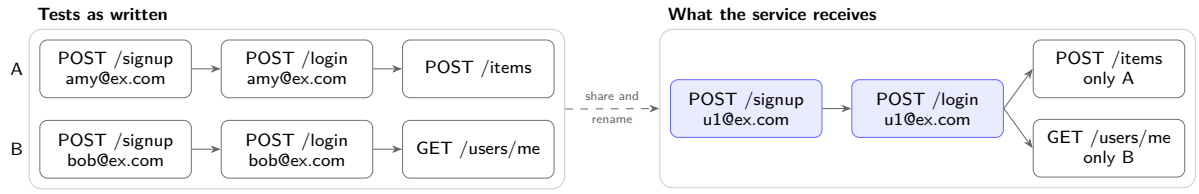


Figure 1: Renaming lets two tests share one signup and one login.

- **Share:** Tests that start with the same setup go into one tree, which runs after one reset, as in Figure 1. A shared request reaches the service once, and the tool hands its answer to the other tests. Two requests are shared when they have the same method, path and body fields, and every value matches in one of three ways: it is equal, it comes from the same earlier answer, or the test made it up. A made up value first appears in a request of that test, such as a fresh email. The tool renames such values, so tests with different emails share one signup. To keep the order safe, the tool predicts from the recorded requests which fields each test reads and writes. A test that reads a field another test writes runs first, or goes to another tree.
- **Replay:** The tool runs the trees with the original test code. Sharing is safe when every test passes exactly the assertions it passes alone. If a test does not, it moves out to run alone and the tool replays.
- **Select:** A budget is the share of tests to keep. An integer program picks the tests that reach the most coverage items, and on a tie the ones with the fewest requests.
- **Boundary values:** In the selected tests, the tool replaces made up values with edges from the API description, such as a maximum length, or else 0, 1 and the empty string. It keeps a value only if every assertion still passes.

4 Evaluation

4.1 Data

We use the test suites of Besjes et al. [3], a study of the LORE group, taken from its public reproduction package (Table 1). A language model amplified one happy path test per API into a suite of Java tests, once with a single agent and once with a team of agents. We use three of its APIs: Spotify and Google Drive, the two largest, and PetStore as a smaller one. Spotify and Google Drive are live cloud services that cannot be reset, and PetStore runs locally in Docker. None of them comes with faults, so black box coverage scores all of them.

Table 1: The APIs and suites of Besjes et al. [3] in the study. Each API has a single agent and a multi agent suite.

API	Runs	Operations	Tests
Spotify	live	89	617 and 617
Google Drive	live	41	189 and 245
PetStore	local	20	108 and 161

Most tests send a single request, and most fail [3]. Only 27 of the 617 Spotify tests and one Google Drive test send a second request. So these suites hold little setup to share, and RQ1 shows how far sharing reaches in suites that a language model amplified. All six suites serve RQ1, RQ2 and RQ3.

4.2 Experiments

Each research question compares a new version of a suite with the full suite it came from. A new version only counts if each of its tests gives the same result as in the full suite, and we leave it unchanged while we measure it.

For RQ1, the tool shares setup in each of the six suites. We compare each suite before and after sharing on requests, resets, run time and black box coverage. As a baseline, a simpler plan shares only requests that are exactly equal, without renaming made up values.

For RQ2, we cut each suite to 75, 50 and 25 percent of its tests and compare the four versions of Table 2. Random keeps tests at random, over 30 draws. Selected keeps the tests that Select picks. Because Select keeps as many coverage items as the budget allows, Selected shows the least coverage a suite of that size can lose, and Random shows what a cut without that care loses. Boundary and Random values start from the Selected tests and change their values: Boundary uses edges, and Random values uses random valid values, over 5 draws. Comparing the two shows whether a gain comes from the edges or from any new value. Random and Selected only drop tests, so their coverage follows from the recorded run of the full suite. Boundary and Random values send new requests, so they need a new run.

Table 2: The four versions of a smaller suite in RQ2, each built at 75, 50 and 25 percent of the tests. None of them shares setup, and black box coverage scores all of them.

Version	Keeps	Values
Random	random tests, 30 draws	as written
Selected	tests that Select picks	as written
Boundary	tests that Select picks	edges
Random values	tests that Select picks	random valid values, 5 draws

For RQ3, we time the tool itself and every suite.

4.3 Metrics

- **Black box coverage lost (main metric, RQ1, RQ2):** Items that the full suite reaches and the new suite misses, counted with RESTCov [9] under eight criteria [1]: paths, operations, parameters, parameter values, request and response content types, status code classes and status codes.
- **Black box coverage won back (RQ2):** Of the coverage items that Selected misses, the number that Boundary reaches, and the same for Random values.
- **Cost (RQ1, RQ3):** Requests and resets per run, where cached answers do not count, and the median run time of five runs.
- **Tests run alone (RQ1):** Tests that cannot share setup, counted by reason, such as failing alone, changing data that another test reads, or losing an assertion when shared.

- **Reduction time and payback (RQ3):** The time the tool takes from the start of Record to the final suite, and that time divided by the run time saved per run.

5 Rough Plan

Phase: Approval and setup
Month: October
Description: Approval, registration in SISA, setup of PetStore in Docker, and access to Spotify and Google Drive.

Phase: Sharing
Month: November
Description: Record, Share and Replay on all six suites, with the simpler plan, and a count of why tests run alone.

Phase: Selection and boundary values
Month: December, January
Description: Select and Boundary values built, and all suites final by 31 January.

Phase: Evaluation
Month: February, March
Description: Coverage and timing runs on all suites. March is a buffer.

Phase: Writing and submission
Month: April, May, June
Description: A full draft by 30 April. By 1 May, a decision with Prof. Demeyer on submitting in June or August. Prof. Demeyer reads the draft in May. Submission and defence in June.

References

[1] Alberto Martin-Lopez, Sergio Segura and Antonio Ruiz-Cortés. Test Coverage Criteria for RESTful Web APIs. A-TEST, 2019.

[2] Tolgahan Bardakci et al. Test Amplification for REST APIs: Using "Out-of-the-Box" Large Language Models. IEEE Software 42(4), 2025.

[3] Jarne Besjes et al. Agentic LLMs for REST API Test Amplification: A Comparative Study Across Cloud Applications. arXiv, 2025. Reproduction package at doi:10.5281/zenodo.22930623.

[4] Hassan Sartaj et al. REST API Testing in DevOps: A Study on an Evolving Healthcare IoT Application. ACM TOSEM 35(6), 2026.

[5] Arash Vahabzadeh et al. Fine-Grained Test Minimization. ICSE, 2018.

[6] Chengpeng Li et al. Reducing Test Runtime by Transforming Test Fixtures. ASE, 2024.

[7] Abdullah Alsharif et al. STICCER: Fast and Effective Database Test Suite Reduction Through Merging of Similar Test Cases. ICST, 2020.

[8] August Shi et al. Evaluating Test-Suite Reduction in Real Software Evolution. ISSTA, 2018.

[9] Tolgahan Bardakci and Serge Demeyer. RESTCov: A Tool for Structural Coverage Analysis of REST APIs. arXiv, 2026.